

Data Lake — Biodiversité des Insectes

Document de référence technique

EFREI 2025-2026 • Projet Final Data Lakes & Data Integration

1. Objectif du projet

Construire un data lake de A à Z pour cartographier la biodiversité des insectes en France, détecter les espèces invasives (frelon asiatique, coccinelle asiatique...) et identifier les meilleurs spots d'observation.

Deux use cases principaux exposés via API :

- Hotspots de biodiversité : richesse spécifique normalisée par cellule H3 (~1 km²)
- Alertes invasives : flagging automatique des espèces de la liste noire INPN

2. Stack technique

Composant	Technologie	Rôle
Stockage Raw	MinIO (S3-compatible)	Fichiers JSON et CSV bruts
Staging / Curated	PostgreSQL + PostGIS	Données transformées et enrichies
Indexation géo	H3 (Uber)	Découpage hexagonal (~1 km²)
Orchestration	Apache Airflow	DAG scheduling ingestion API
API Gateway	FastAPI (Python)	Exposition des endpoints
Conteneurisation	Docker + Compose	Stack complète en local
ML	scikit-learn	Classifieur invasif/non-invasif

3. Sources de données

3.1 iNaturalist API (source temps réel)

URL de base :

```
https://api.inaturalist.org/v1/observations
```

Paramètres clés pour les insectes en France :

```
taxon_id=47158 # Insecta
place_id=6753 # France métropolitaine
quality_grade=research,needs_id
per_page=200 # max par appel
```

⚠ L'API est paginée. Utilise le paramètre `page=` et arrête-toi quand `total_results` est atteint. Respecte le `rate limit` : max 100 requêtes/minute.

3.2 GBIF Dataset (source fichier)

Télécharger le dataset CSV depuis :

| <https://www.gbif.org/occurrence/download>

Filtres à appliquer sur le portail GBIF avant export :

- Taxon : Insecta (classe)
- Pays : France
- Année : 2010 → présent (pour éviter les données trop anciennes)
- Format : CSV (occurrence.txt dans le ZIP)

⚠ Le ZIP GBIF peut faire plusieurs Go. Filtre bien avant de télécharger. Le fichier `occurrence.txt` est *tab-separated*, pas *comma-separated*.

4. Architecture du data lake

4.1 Zone Raw (MinIO)

Stockage brut sans transformation. Deux buckets :

- `raw-inaturalist/` → fichiers JSON horodatés (un fichier par run Airflow)
- `raw-gbif/` → fichier `occurrence.txt` original

Convention de nommage :

| `raw-inaturalist/YYYY-MM-DD/observations_HH-MM.json`

4.2 Zone Staging (PostgreSQL)

Table principale : `staging.occurrences`

Colonne	Type	Description
<code>id</code>	VARCHAR	Identifiant source (iNat ID ou GBIF ID)
<code>species_name</code>	VARCHAR	Nom scientifique de l'espèce
<code>latitude</code>	FLOAT	Latitude WGS84
<code>longitude</code>	FLOAT	Longitude WGS84
<code>observed_on</code>	DATE	Date d'observation
<code>quality_grade</code>	VARCHAR	<code>research</code> / <code>needs_id</code>
<code>source</code>	VARCHAR	<code>inaturalist</code> / <code>gbif</code>
<code>raw_payload</code>	JSONB	Payload original complet

4.3 Zone Curated (PostgreSQL)

Table 1 : `curated.species_richness_h3`

Colonne	Type	Description
h3_cell	VARCHAR	Identifiant hexagone H3 résolution 7
species_count	INT	Nombre d'espèces uniques (richesse brute)
obs_count	INT	Nombre total d'observations
richness_normalized	FLOAT	species_count / log(1 + obs_count)
richness_percentile	FLOAT	Percentile 0-100 (100 = hotspot)
lat_centroid	FLOAT	Latitude centre de la cellule
lon_centroid	FLOAT	Longitude centre de la cellule
last_observed	DATE	Date de la dernière observation

Table 2 : curated.invasive_hotspots

Colonne	Type	Description
h3_cell	VARCHAR	Identifiant hexagone H3
species_name	VARCHAR	Nom scientifique de l'espèce
invasive_risk	VARCHAR	high / medium / low
alert_count	INT	Nombre d'observations de cette espèce
first_seen	DATE	Première observation
last_seen	DATE	Dernière observation (la plus récente)
lat_centroid	FLOAT	Latitude approximative du hotspot
lon_centroid	FLOAT	Longitude approximative du hotspot

5. Endpoints API (FastAPI)

Endpoint	Méthode	Description
/health	GET	Statut des services (MinIO, PostgreSQL, Airflow)
/stats	GET	Métriques : nb occurrences par zone, dernière ingestion
/raw	GET	Liste des fichiers dans MinIO (raw-inaturalist/, raw-gbif/)
/staging	GET	Occurrences paginées depuis staging.occurrences
/curated/hotspots	GET	Richesse spécifique H3, triée par percentile desc
/curated/invasives	GET	Alertes invasives, filtrables par risk=high/medium/low
/ingest	POST	Ingestion manuelle d'observations JSON → pipeline complet
/ingest_fast	POST	Ingestion optimisée (+30% perf, vectorisée NumPy)

Exemple de payload /ingest

```
POST /ingest
```

```
{
  "data": {
    "observations": [
      { "species_name": "Vespa velutina", "latitude": 48.85,
"longitude": 2.35, "observed_on": "2025-06-01" }
    ]
  }
}
```

6. Intégration Claude Haiku 4.5

6.1 Choix du modèle

Modèle utilisé : claude-haiku-4-5 (API string officiel). C'est le modèle Anthropic le plus rapide et économique de la gamme Claude 4.5, idéal pour des traitements en volume comme l'enrichissement de descriptions d'espèces ou la génération de résumés d'alertes invasives.

Caractéristique	Valeur
API string	claude-haiku-4-5
Contexte max	200 000 tokens
Prix input	\$1 / million de tokens
Prix output	\$5 / million de tokens
Use case ici	Enrichissement descriptions + résumés alertes

6.2 Exemple d'appel API

```
import anthropic

client = anthropic.Anthropic() # ANTHROPIC_API_KEY en variable d'env

def enrich_species_description(species_name: str) -> str:
    """Génère une description courte de l'espèce pour l'endpoint
    /curated."""
    message = client.messages.create(
        model="claude-haiku-4-5",
        max_tokens=256,
        system="Tu es un entomologiste. Réponds en 2 phrases max, en
français.",
        messages=[{
            "role": "user",
            "content": f"Décris brièvement {species_name} et pourquoi
elle est invasive."
        }]
    )
    return message.content[0].text
```

⚠ Ne jamais hardcoder la clé API. Utiliser une variable d'environnement `ANTHROPIC_API_KEY` dans le fichier `.env Docker`.

6.3 Use cases dans le pipeline

- Enrichissement curated : générer une description pour chaque espèce invasive détectée
- Résumé d'alerte : synthétiser les observations récentes d'une zone en langage naturel
- Optionnel — aide à la classification : suggérer un niveau de risque pour une espèce inconnue

7. Structure Docker

7.1 Services docker-compose.yml

Service	Image	Port
minio	minio/minio	9000 (API), 9001 (console)
postgres	postgis/postgis:15-3.4	5432
airflow	apache/airflow:2.9.1	8080
api	build: ./api (FastAPI)	8000

7.2 Variables d'environnement (.env)

```
MINIO_ROOT_USER=minioadmin
MINIO_ROOT_PASSWORD=minioadmin
POSTGRES_USER=insect_user
POSTGRES_PASSWORD=insect_pass
POSTGRES_DB=insect_lake
ANTHROPIC_API_KEY=sk-ant-... # ta clé Anthropic
INATURALIST_API_KEY=... # optionnel, augmente le rate limit
```

⚠ Ajouter `.env` au `.gitignore` immédiatement. Ne jamais commit de clés API.

8. Optimisation /ingest_fast

Objectif : +30% de performance par rapport à `/ingest`. Les trois leviers principaux :

Levier 1 — Vectorisation H3 avec NumPy

Remplacer le `.apply()` ligne par ligne par un appel vectorisé :

```
# Lent (O(n) appels Python)
df['h3_cell'] = df.apply(lambda r: h3.geo_to_h3(r.lat, r.lon, 7),
axis=1)
```

```
# Rapide (vectorisé)
import numpy as np
geo_to_h3_vec = np.vectorize(lambda lat, lon: h3.geo_to_h3(lat, lon, 7))
df['h3_cell'] = geo_to_h3_vec(df['latitude'].values,
df['longitude'].values)
```

Levier 2 — Écritures async en base

Utiliser `asyncpg` + `asyncio` pour paralléliser les INSERT en PostgreSQL.

Levier 3 — Mise en cache de la liste noire

Charger `INVASIVE_SPECIES` en mémoire au démarrage (`lru_cache` ou variable globale), pas à chaque appel.

⚠ Documenter les temps d'exécution pour 1 élément et 100 éléments dans le README. C'est un critère d'évaluation explicite du niveau avancé.

9. DAG Airflow

Structure du DAG principal (`ingest_inaturalist_dag.py`) :

| `fetch_api` → `save_to_raw` → `validate_staging` → `transform_curated` → `notify`

- Scheduling : `@hourly` pour iNaturalist, `@daily` pour la transformation Curated
- XCom : passer le chemin du fichier Raw entre `fetch_api` et `save_to_raw`
- Retry : 3 tentatives, délai 5 minutes (pour les timeouts API)

⚠ Tester le DAG manuellement avec `airflow dags trigger ingest_inaturalist` avant d'activer le scheduling automatique.

10. Checklist de livraison

Item	Critère prof	Statut
Zone Raw (MinIO)	Architecture data lake	☐
Zone Staging (PostgreSQL)	Architecture data lake	☐
Zone Curated (H3 + invasives)	Architecture data lake	☐
Scripts de transformation robustes	Gestion exceptions + logs	☐
DAG Airflow avec scheduling	Pipeline d'intégration	☐
Endpoints <code>/raw</code> <code>/staging</code> <code>/curated</code>	API Gateway complète	☐

Item	Critère prof	Statut
Endpoints /health /stats	API Gateway complète	☐
Endpoint /ingest	Niveau avancé	☐
Endpoint /ingest_fast (+30%)	Niveau avancé	☐
Benchmarks documentés (1 et 100 éléments)	Documentation perf	☐
README avec procédure de build	Documentation	☐
Code commenté	Qualité code	☐
Dépôt GitHub propre	Livrable final	☐